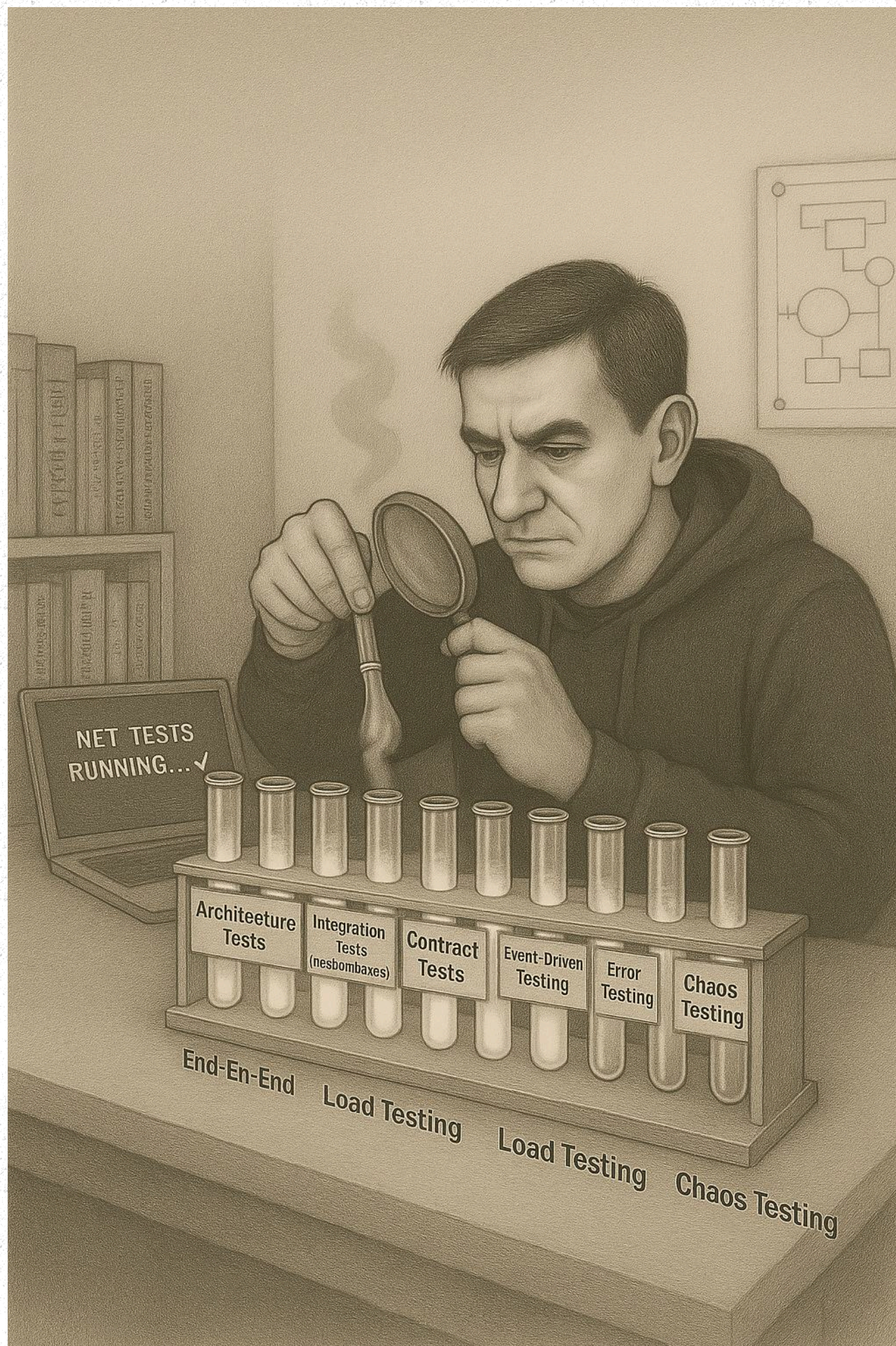


10 Secret Testing Techniques in .NET that senior developers know That juniors and middles don't know about



01

Architecture Tests

- Verify that your layers, projects, and classes follow your architectural rules.
- Ensure that your system's design rules are enforced and that your code follows the architecture you've set up.

```
using NetArchTest.Rules;
using Xunit;

namespace Shipments.Tests.Architecture;

public class CleanArchitectureTests
{
    private const string Domain = "Shipments.Domain";
    private const string Application = "Shipments.Application";
    private const string Infrastructure = "Shipments.Infrastructure";
    private const string Api = "Shipments.Api";

    [Fact]
    public void Domain_Should_Not_Depend_On_Application_Infrastructure_Or_Api()
    {
        var result = Types
            .InAssembly(DomainAssembly)
            .That()
            .ResideInNamespace(Domain)
            .ShouldNot()
            .HaveDependencyOn(Application)
            .AndShouldNot()
            .HaveDependencyOn(Infrastructure)
            .AndShouldNot()
            .HaveDependencyOn(Api)
            .GetResult();

        Assert.True(result.IsSuccessful, FormatFailure(result));
    }
}
```



02

Integration Tests with TestContainers

- Spin up real databases, queues, or Cache in disposable Docker containers.
- Test real integrations to catch any issues that may arise when these components interact with each other.

```
[Collection("ShippingTests")]
public class CreateShipmentTests(CustomWebApplicationFactory factory) : IAsyncLifetime
{
    public Task InitializeAsync() => Task.CompletedTask;

    public async Task DisposeAsync() => Task.CompletedTask;

    [Fact]
    public async Task CreateShipment_ShouldSucceed_WhenRequestIsValid()
    {
        // Arrange
        var request = new CreateShipmentRequest(...);

        // Act
        var httpResponse = await factory.HttpClient.PostAsJsonAsync("/api/shipments", request);
        var shipmentResponse = await httpResponse.Content.ReadFromJsonAsync<CreateShipmentResponse>();

        // Assert
        var expectedResponse = new ShipmentResponse(shipmentResponse.Number, "12345",
            address, "Modern Shipping", "test@mail.com", ShipmentStatus.Created, items);

        shipmentResponse
            .Should()
            .BeEquivalentTo(expectedResponse);
    }
}
```



Contract Tests

- Ensure that API or message contracts between services remain compatible.
- Avoid breaking your consumers when refactoring or adding new fields.

```
[Fact]
public async Task CreateShipment_ShouldSucceed_WhenRequestIsValid()
{
    // Arrange
    var request = new CreateShipmentRequest(...);

    // Act
    var httpResponse = await factory.HttpClient.PostAsJsonAsync("/api/shipments", request);
    var shipmentResponse = (await httpResponse.Content
        .ReadFromJsonAsync<ShipmentResponse>(_jsonSerializerOptions))!;

    // Assert
    httpResponse.StatusCode.Should().Be(HttpStatusCode.OK);

    shipmentResponse.Should().BeEquivalentTo(new ShipmentResponse(
        shipment.Number,
        shipment.OrderId,
        shipment.Address,
        shipment.Carrier,
        shipment.ReceiverEmail,
        shipment.Status,
        shipment.Items.Select(i => new ShipmentItemResponse(i.Product, i.Quantity)).ToList()
    ));
}
```



04

Event-Driven Testing

- Validate event publishing, consuming, and idempotency in distributed systems.
- Confirm your events trigger expected outcomes.

```
var isEventPublished = await massTransitHarness.Published.Any<ShipmentCreatedEvent>();
isEventPublished.Should().BeTrue();

var consumerHarness = massTransitHarness.GetConsumerHarness<ShipmentCreatedConsumer>();

var isEventConsumed = await consumerHarness.Consumed.Any<ShipmentCreatedEvent>(x =>
{
    var message = x.Context.Message;

    return message.OrderId == shipmentResponse.OrderId &&
        message.Number == shipmentResponse.Number &&
        message.Carrier == shipmentResponse.Carrier &&
        message.ReceiverEmail == shipmentResponse.ReceiverEmail &&
        message.Status == Shared.Models.ShipmentStatus.Created &&
        message.Address.City == shipmentResponse.Address.City &&
        message.Address.Street == shipmentResponse.Address.Street &&
        message.Address.Zip == shipmentResponse.Address.Zip;
});

isEventConsumed.Should().BeTrue();

await massTransitHarness.Stop();
```



05

Error Testing

- Simulate faults and exceptions to confirm resilience and proper error handling.
- A system is only as strong as its failure paths.

```
[Fact]
public async Task CreateShipment_ShouldReturnConflict_WhenShipmentForOrderIsAlreadyCreated()
{
    // Arrange
    var request = new CreateShipmentRequest(...);

    // Act
    await factory.HttpClient.PostAsJsonAsync("/api/shipments", request);

    var httpResponse = await factory.HttpClient.PostAsJsonAsync("/api/shipments", request);
    var validationResult = await httpResponse.Content.ReadFromJsonAsync<ValidationProblemDetails>();

    // Assert
    httpResponse.StatusCode.Should().Be(HttpStatusCode.Conflict);
    validationResult?.Errors.Should().HaveCount(1);

    var error = validationResult?.Errors.FirstOrDefault();
    error?.Key.Should().Be("Shipment for order '12345' is already created");
    error?.Value.First().Should().Be("A conflict error has occurred.");
}
```



06

Snapshot Testing

- Capture structured outputs (GraphQL, JSON, XML, etc.) and detect unwanted changes.
- Perfect for APIs and frontends for testing large responses.

```
[Fact]
public async Task CreateShipment_ShouldSucceed_WhenRequestIsValid()
{
    // Arrange
    var request = new CreateShipmentRequest(...);

    // Act
    var httpResponse = await factory.HttpClient.PostAsJsonAsync("/api/shipments", request);
    var shipmentResponse = (await httpResponse.Content
        .ReadFromJsonAsync<ShipmentResponse>(_jsonSerializerOptions))!;

    // Assert
    httpResponse.StatusCode.Should().Be(HttpStatusCode.OK);

    // Replace complex assertions with single line of code
    await Verify(shipmentResponse);
}
```

GetShipmentByNumberTests.GetShipmentByNumber_ShouldReturnShip...

Location Pane x

GetShipmentByNumberTests.GetShipmentByNumber_ShouldReturnShipment_WhenShipmentExists.received.txt

1

```
{
  Number: 93350877,
  OrderId: 12345,
  Address: {
    Street: Amazing st. 5,
    City: New York,
    Zip: 127675
  },
  Carrier: Modern Shipping,
  ReceiverEmail: test@mail.com,
  Items: [
    {
      Product: Samsung Electronics,
      Quantity: 1
    }
  ]
}
```

GetShipmentByNumberTests.GetShipmentByNumber_ShouldReturnShipment_WhenShipmentExists.verified.txt

2

```
{
  Number: 14331084,
  OrderId: 12345,
  Address: {
    Street: Amazing st. 5,
    City: New York,
    Zip: 127675
  },
  Carrier: Modern Shipping,
  ReceiverEmail: test@mail.com,
  Items: [
    {
      Product: Samsung Electronics,
      Quantity: 1
    }
  ]
}
```



07

Behavioral Testing

- Verify system behavior from the user's or domain's perspective using real scenarios.
- Focus on what the system should do.

```
Scenario: VIP discount for large order
  Given a VIP customer with total order amount 1000
  Then the discount should be 15 percent
```

```
[Binding]
public class DiscountSteps
{
    private Order _order = null!;

    [Given("a VIP customer with total order amount {int}")]
    public void GivenVipCustomer(int amount)
    {
        _order = new Order(amount, isVip: true);
    }

    [Then("the discount should be {int} percent")]
    public void ThenTheDiscountShouldBe(int expected)
    {
        var discount = new DiscountService().Calculate(_order);

        discount.Should().Be(expected);
    }
}
```



08

End-to-End (E2E) Testing

- Test complete workflows across services, databases, and UI.
- Ensures all parts of your system communicate correctly in real-world conditions.

```
[Fact]
public async Task User_Can_Create_Shipment_From_Ui()
{
    using var playwright = await Playwright.CreateAsync();
    var browser = await playwright.Chromium.LaunchAsync(new BrowserTypeLaunchOptions
    {
        Headless = true
    });
    var page = await browser.NewPageAsync();

    await page.GotoAsync("https://localhost:5001/shipments/create");
    await page.FillAsync("#orderId", "ORDER-123");
    await page.FillAsync("#address", "Amazing st. 5, New York");
    await page.FillAsync("#carrier", "Modern Shipping");
    await page.ClickAsync("#submit");

    await page.WaitForURLAsync("*/shipments/ORDER-123");
    var status = await page.TextContentAsync("[data-test-id=\"shipment-status\"]");

    status.Should().Be("Created");
}
```



09

Load Testing

- Measure how your system performs under high traffic, concurrency, and stress.
- Helps identify bottlenecks before users do.

```
var httpClient = Http.CreateDefaultClient();

var scenario = Scenario.Create("user_login_and_get_profile_scenario",
    async context =>
    {
        var step1 = await Step.Run("login", context, () => LoginStep(httpClient));

        var token = step1.Payload.Value;

        var step2 = await Step.Run("get_profile", context, () => GetProfileStep(httpClient, token));

        return step2;
    })
    .WithoutWarmUp()
    .WithLoadSimulations(
        Simulation.Inject(rate: 10,
            interval: TimeSpan.FromSeconds(1),
            during: TimeSpan.FromSeconds(30))
    );
```



Chaos & Resilience Testing

- Inject random failures to validate fault tolerance and recovery mechanisms.
- Real systems must survive the unexpected failures.

```
[Fact]
public async Task Api_Should_Recover_When_Temporary_Failures_Occur()
{
    var failures = 0;

    var policy = Policy
        .Handle<HttpRequestException>()
        .RetryAsync(3);

    var result = await policy.ExecuteAsync(async () =>
    {
        failures++;

        if (failures < 3)
        {
            throw new HttpRequestException("Chaos: simulated outage");
        }

        return "ok";
    });

    failures.Should().Be(3);
    result.Should().Be("ok");
}
```



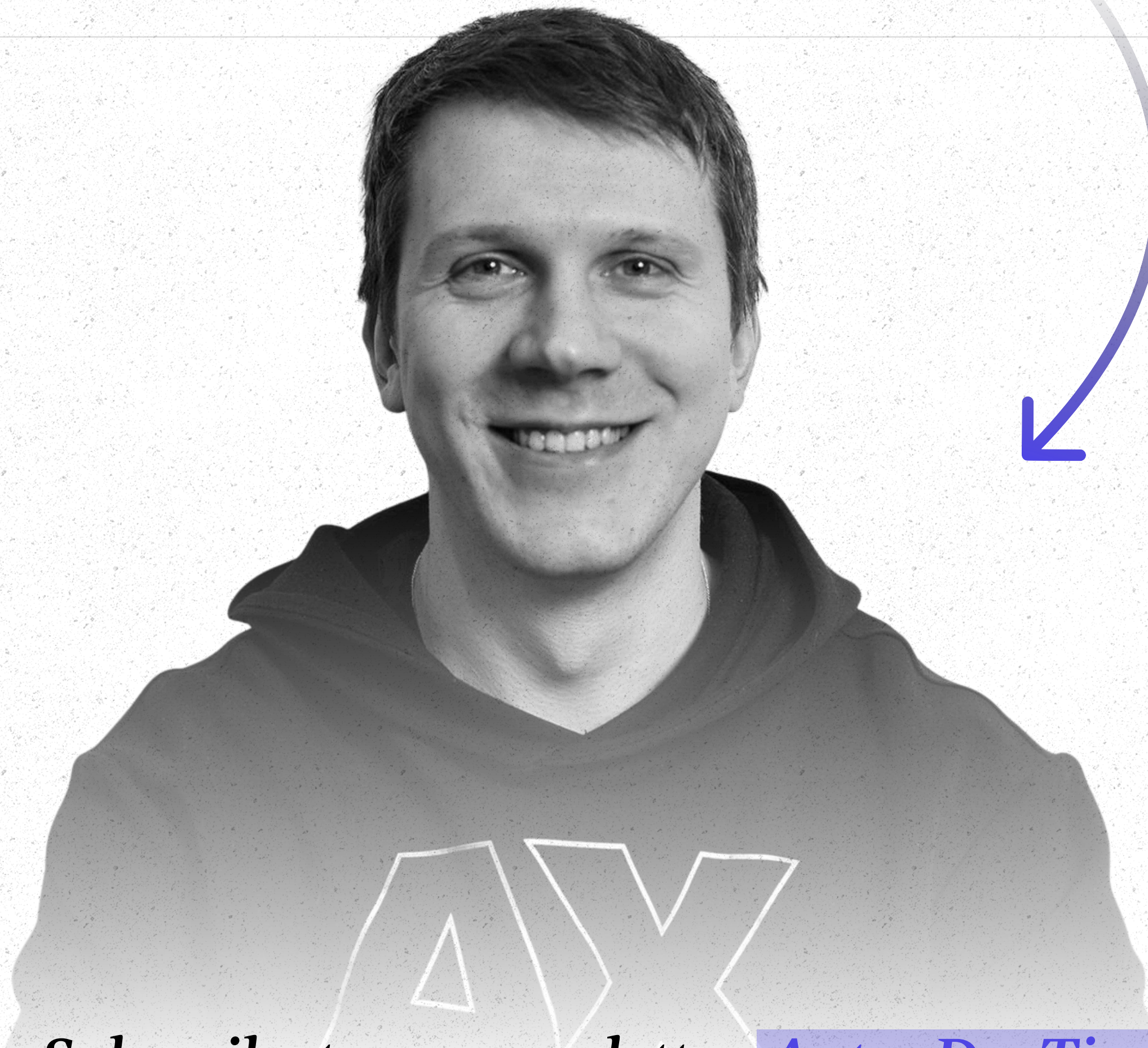
Find this useful?



Repost if you found this insightful



Save for future inspiration



Subscribe to my newsletter, [AntonDevTips](#),
for more insights on .NET and Architecture